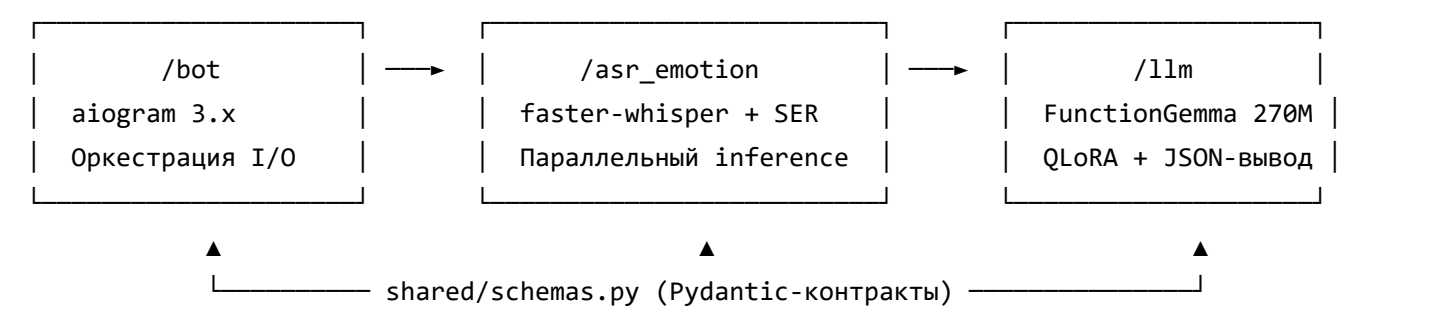


SpeechToPlan — материалы для защиты

Telegram-ассистент-планировщик. Принимает голос/текст, локально транскрибирует речь, параллельно классифицирует эмоцию говорящего и через локальную LLM (FunctionGemma-270m-it) превращает фразу в строгий JSON-объект задачи.

1. Архитектурное разделение модулей

Проект построен по принципу **трёх изолированных компонент**, каждая со своими зависимостями и собственным контрактом ввода/вывода:



Модуль	Ответственность	Не знает про
/bot	приём апдейтов Telegram, скачивание ogg, форматирование ответа	модели, токенизаторы, веса
/asr_emotion	speech-to-text + speech emotion recognition	Telegram, JSON-схему задач
/llm	генерация валидного JSON по тексту + эмоции	как получено аудио

Связь между модулями — только через `shared.schemas` :
`EnrichedUtterance (text + emotion) → PlannerTask (готовая задача)`.
Это позволяет:

- независимо обновлять и обучать каждую часть;

- тестировать модули в изоляции (LLM-юнит-тест не требует ASR);
- собирать в Docker-образ только нужный набор зависимостей.

2. Обоснование «Варианта Б»: параллельные ASR + SER

Что такое «Вариант Б»

Аудио параллельно скормливается **двум независимым моделям**:

- `faster-whisper-tiny` (CTranslate2, INT8) — текст;
- `wav2vec2-base-superb-er` (HuggingFace, fp16) — эмоция;

а затем результаты склеиваются: [Эмоция: `anxious` | URGENT] Текст:

Альтернативы, которые отвергнуты

1. **Дообучить Whisper выдавать эмоции** в спецтокенах. Архитектурная проблема: декодер Whisper заточен на текстовые токены и BPE русского словаря; добавление эмоций требует пересборки токенизатора и существенного объёма размеченных данных (аудио, текст, эмоция), которых в open-source мало. Ожидаемая деградация WER.
2. **Multi-task encoder (общий wav2vec2 → две головы)**. Нужен совместный тренинг и единый домен данных, плюс CTC-голова для ASR качеством сильно слабее Whisper.

Почему Вариант Б оптимален именно для нашего железа

- Whisper работает в **CTranslate2** (C++): он отпускает GIL, поэтому Python-поток для `wav2vec2` действительно идёт **параллельно**, а не псевдо-параллельно.
- Каждая модель занимает **<1 ГБ VRAM** в режиме INT8/fp16 — обе влезают рядом с LLM в 4 ГБ карты.
- Один декод аудио (`ffmpeg` → `np.float32 16 кГц`), который шарится между головами.
- Полная независимость моделей: можно менять SER (например, на HuBERT) без перетренировки ASR и наоборот.

Реализация — `asr_emotion/inference.py` :

```
fut_asr = self.pool.submit(self.asr.transcribe, audio_path)
fut_ser = self.pool.submit(self.ser.predict, audio_np)
asr_res, ser_res = fut_asr.result(), fut_ser.result()
```

3. ML-обучение

3.1 Дообучение Whisper-tiny (LoRA + WER)

Скрипт: `asr_emotion/training/train_whisper_lora.py` .

- Базовая модель грузится в `fp16` , целевые модули LoRA: `q_proj` , `v_proj` .
- Включён `gradient_checkpointing` и отключён `use_cache` (обязательное парное условие, иначе `backward` сломается).
- Метрика — стандартный **WER** на `validation split`, считается через `evaluate.load("wer")` и `predict_with_generate=True` .
- Сохраняется только LoRA-адаптер (~5 МБ) — базовые веса не дублируем.

3.2 QLoRA для FunctionGemma + Grid Search

Скрипт: `llm/training/qlora_grid_search.py` .

Почему QLoRA, а не full fine-tune.

Full FT 270M-модели в `fp16` — ~1.1 ГБ только под веса + ещё столько же под градиенты и состояние оптимизатора. На RTX 3050 Ti это OOM сразу. QLoRA даёт:

- база квантована в **NF4** (4 бита/вес, ~250 МБ),
- градиенты считаются только для LoRA-адаптеров (~0.5–1% параметров),
- оптимизатор `paged_adamw_8bit` сбрасывает состояния на CPU/swap.

Итого свободно <2 ГБ VRAM — достаточно для `batch=2` + `grad-accum=8`.

Почему Grid Search, а не k-fold кросс-валидация.

- Грубая оценка вычислительной стоимости: k-fold с `k=5` умножает обучение **в 5 раз** на каждую точку гиперпараметров. При 12 точках сетки (`3 lr × 2 rank × 2 alpha`) и ~10 минут на запуск это **10 часов** вместо ~2.
- Размер нашего датасета `текст→JSON` небольшой и легко делится `train/val` без риска «съесть» хвост распределения — однократный `hold-out` валиден.
- Цель курсовой — показать **академический алгоритм перебора**, а не «выжать» последний 0.1% качества; Grid Search прозрачен, легко рисуется таблица «параметры → `loss`».
- Lite-альтернативу (k-fold) при необходимости можно включить заменой одной функции — обвязка `trainer`-а уже модульная.

Сетка по умолчанию:

- `learning_rate` ∈ {5e-5, 1e-4, 3e-4}
- `lora_rank` ∈ {8, 16}
- `lora_alpha` ∈ {16, 32}

Для каждой комбинации:

1. база перезагружается «с чистого листа» (адаптеры между точками не наслаиваются);
2. обучение N эпох с `eval_strategy="epoch"` ;
3. `eval_loss` лучшего чекпоинта запоминается;
4. результаты пишутся в `grid_summary.json` ;
5. лучший адаптер копируется в `best_adapter/` и подключается на инференсе через переменную окружения `LLM_ADAPTER_PATH` .

3.3 Сводка по бюджету VRAM (RTX 3050 Ti, 4 ГБ)

Этап	Модель	Режим	Approx VRAM
Inference	whisper-tiny	INT8 (CT2)	~150 МБ
Inference	wav2vec2-base SER	fp16	~250 МБ
Inference	FunctionGemma 270M	NF4 + double quant	~250 МБ
Training	whisper-tiny + LoRA	fp16 + ckpt + bs=4	~3.0 ГБ
Training	Gemma + QLoRA	NF4 + ckpt + bs=2	~2.5 ГБ

4. Контракт данных

- `EnrichedUtterance.to_prompt()` — единственный способ собрать строку для LLM.
- `PlannerTask` — Pydantic-схема, валидирует всё, что пришло из LLM (если модель выдала мусор — `inference.to_task` ловит и собирает fallback-задачу).

5. Тезисы для устной части

1. Это не «обёртка над OpenAI» — все три модели локальные.
2. Параллельный SER даёт информацию, которой текст в принципе не содержит:
фразу «ну ок, сделаю» можно сказать спокойно или с тревогой — это разная срочность.

3. Grid Search и k-fold — взаимодополняющие техники; выбор продиктован вычислительным бюджетом, а не «k-fold не нужен».
4. QLoRA — это композиция трёх независимых трюков: NF4-квантование, LoRA-адаптеры, paged optimizer. Любой из них в отдельности 4 ГБ не хватит.